

ÜrTC — Travel Companion

Computer Program Source Code Deposit

First 25 and Last 25 Pages

Title of Work: ÜrTC (Travel Companion Application), Version 1.1

Author / Claimant: Cave Core Dynamics LLC

Year of Completion: 2026

Total source pages (54 lines/page): 508

Trade secret material (API credentials) has been redacted
in accordance with U.S. Copyright Office Circular 61.

Prepared: July 17, 2026

FIRST 25 PAGES

```
=====
FILE: index.tsx
=====
```

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import { LanguageProvider } from './i18n/context';

const rootElement = document.getElementById('root');
if (!rootElement) {
  throw new Error("Could not find root element to mount to");
}

const root = ReactDOM.createRoot(rootElement);
root.render(
  <React.StrictMode>
    <LanguageProvider>
      <App />
    </LanguageProvider>
  </React.StrictMode>
);
```

```
=====
FILE: App.tsx
=====
```

```
import React, { useState, useEffect } from 'react';
import { Plane, Building2, Moon, Sun, Mic, Info, Notebook, WifiOff, Home, X, Clock, Calendar, Shield, Globe, Map, MapPinOff, Settings } from 'lucide-react';
import { useSwipeable } from 'react-swipeable';
import { HomeView } from './components/HomeView';
import { Tab, BudgetItem, UserTier, Note, Theme, Pass, Flight, UserAccount } from './types';
import { FlightView } from './components/FlightView';
import { CityView } from './components/CityView';
import { ApolloView } from './components/ApolloView';
import { ItineraryView } from './components/ItineraryView';
import { SocialView } from './components/SocialView';
import { EnhancedApolloDogIcon } from './components/ApolloDog';
import { ApolloLive } from './components/ApolloLive';
import { DynamicIsland } from './components/DynamicIsland';
import { AboutView } from './components/AboutView';
import { DiamondTutorialOverlay } from './components/DiamondTutorialOverlay';
import { TutorialOverlay } from './components/TutorialOverlay';
import { getActiveUser, setActiveUser } from './services/authService';
import { auth, db } from './services/firebaseClient';
import { onAuthStateChanged } from 'firebase/auth';
import { doc, getDoc } from 'firebase/firestore';
import { fetchRealFlights } from './services/apiService';
import { useLanguage } from './i18n/context';
import { AuthGate } from './components/auth/AuthGate';

const SplashScreen: React.FC<{ onComplete: () => void }> = ({ onComplete }) => {
  useEffect(() => {
    const timer = setTimeout(() => {
      onComplete();
    }, 3000);
  }, []);
  return (
    <div>
      <AuthGate />
      <DiamondTutorialOverlay />
      <TutorialOverlay />
      <AboutView />
      <DynamicIsland />
      <ApolloLive />
      <EnhancedApolloDogIcon />
      <SocialView />
      <ItineraryView />
      <ApolloView />
      <CityView />
      <FlightView />
      <HomeView />
    </div>
  );
};
```

```

    }, 2200);
    return () => clearTimeout(timer);
  }, [onComplete]);

  return (
    <div className="fixed inset-0 z-[100] bg-brand-ink flex flex-col items-center justify-center overflow-hidden">
      { /* Deep ambient glow */ }
      <div className="absolute inset-0 pointer-events-none">
        <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 w-[500px] h-[500px] bg-brand-orange/10 rounded-full blur-[120px] animate-pulse" />
        <div className="absolute top-1/3 left-1/4 w-[300px] h-[300px] bg-brand-blue/8 rounded-full blur-[100px]" />
      </div>

      <div className="relative z-10 flex flex-col items-center gap-8 animate-blur-in">
        { /* Apollo ring */ }
        <div className="relative">
          <div className="absolute inset-[-8px] rounded-full bg-gradient-to-br from-brand-orange/50 to-brand-blue/30 blur-lg animate-pulse" />
          <div className="relative w-28 h-28 rounded-full border-2 border-brand-orange/60 p-0.5 shadow-2xl">
            
          </div>
        </div>

        { /* Wordmark */ }
        <div className="text-center space-y-2">
          <h1 className="font-display text-7xl font-bold tracking-tight text-white leading-none">
            Ür<span className="gradient-text">TC</span>
          </h1>
          <p className="text-white/40 text-sm font-medium tracking-[0.3em] uppercase">Travel Evolved</p>
        </div>

        { /* Powered by badge */ }
        <div className="px-4 py-1.5 rounded-full border border-white/10 bg-white/5 backdrop-blur-md">
          <p className="text-[11px] font-semibold text-white/50 tracking-wider uppercase">Powered by Apollo AI · Cave Core Dynamics™</p>
        </div>
      </div>
    </div>
  );
};

const TAB_ORDER = [Tab.Home, Tab.Flights, Tab.Explore, Tab.Itinerary];

const AppContent: React.FC = () => {
  const { t } = useLanguage();
  const [user, setUser] = useState<UserAccount>(getActiveUser());
  const [activeTab, setActiveTab] = useState<Tab>(Tab.Home);

```

```

const [direction, setDirection] = useState(0);
const [showSplash, setShowSplash] = useState(true);
const [budgetItems, setBudgetItems] = useState<BudgetItem[]>([]);
const [budgetLimit, setBudgetLimit] = useState(2500);
const [notes, setNotes] = useState<Note[]>([]);
const [passes, setPasses] = useState<Pass[]>([]);
const [theme, setTheme] = useState<Theme>('dark');
const [textSize, setTextSize] = useState<'sm' | 'base' | 'lg'>('base');
const [exploreCity, setExploreCity] = useState("Atlanta");
const [showLive, setShowLive] = useState(false);
const [isOffline, setIsOffline] = useState(!navigator.onLine);
const [trackedActivity, setTrackedActivity] = useState<Flight | null>(null);
const [alertMessage, setAlertMessage] = useState<string | undefined>();
const [showTerms, setShowTerms] = useState(() => !localStorage.getItem('urtc_terms_accepted'));
const [showDiamondTutorial, setShowDiamondTutorial] = useState(false);
const [runTour, setRunTour] = useState(false);
const [locationDenied, setLocationDenied] = useState(false);

// Global Geolocation Request
useEffect(() => {
  if (!showTerms) {
    if ("geolocation" in navigator) {
      navigator.geolocation.getCurrentPosition(
        (position) => {
          setLocationDenied(false);
          if (exploreCity === "Atlanta" && window.google) {
            const { latitude, longitude } = position.coords;
            const geocoder = new window.google.maps.Geocoder();
            geocoder.geocode({ location: { lat: latitude, lng: longitude } }, (results: any[], status
: any) => {
              if (status === 'OK' && results[0]) {
                const cityComp = results[0].address_components.find((c: any) => c.types.includes(
'locality'));
                const stateComp = results[0].address_components.find((c: any) => c.types.includes
('administrative_area_level_1'));
                const countryComp = results[0].address_components.find((c: any) => c.types.includ
es('country'));

                let detectedCity = '';
                if (cityComp) detectedCity += cityComp.long_name;
                if (stateComp && countryComp?.short_name === 'US') detectedCity += `, ${stateComp
.short_name}`;
                else if (countryComp) detectedCity += `, ${countryComp.short_name}`;

                if (detectedCity) setExploreCity(detectedCity);
              }
            });
          }
        },
        (error) => {
          if (error.code === error.PERMISSION_DENIED) {
            setLocationDenied(true);
          }
        }
      );
    }
  }
}

```

```
    );
  } else {
    setLocationDenied(true);
  }
}
}, [showTerms]);

// Trigger Diamond Tutorial
useEffect(() => {
  if (user.tier === UserTier.Diamond && !localStorage.getItem('urtc_diamond_tutorial_seen')) {
    setShowDiamondTutorial(true);
  }
}, [user.tier]);

const handleCloseDiamondTutorial = () => {
  setShowDiamondTutorial(false);
  localStorage.setItem('urtc_diamond_tutorial_seen', 'true');
};

// Firebase Auth Listener
useEffect(() => {
  const unsubscribe = onAuthStateChanged(auth, async (firebaseUser) => {
    if (!firebaseUser) {
      const guestUser: UserAccount = {
        id: 'guest',
        username: 'Guest',
        passwordHash: '',
        tier: UserTier.Guest,
        savedTrips: []
      };
      setUser(guestUser);
      setActiveUser(guestUser);
    } else {
      // Fetch profile
      const userDoc = await getDoc(doc(db, 'users', firebaseUser.uid));
      const profile = userDoc.exists() ? userDoc.data() : null;

      const activeUser: UserAccount = {
        id: firebaseUser.uid,
        username: profile?.username || firebaseUser.email?.split('@')[0] || 'Traveler',
        passwordHash: '',
        email: firebaseUser.email || undefined,
        tier: profile?.tier as UserTier || UserTier.Free,
        savedTrips: []
      };
      setUser(activeUser);
      setActiveUser(activeUser);
    }
  });
});

return () => unsubscribe();
```

```
}, []);

// Refresh user state from storage on mount
useEffect(() => {
  setUser(getActiveUser());
}, []);

// Handle Offline Mode
useEffect(() => {
  const handleOnline = () => setIsOffline(false);
  const handleOffline = () => setIsOffline(true);
  window.addEventListener('online', handleOnline);
  window.addEventListener('offline', handleOffline);
  return () => {
    window.removeEventListener('online', handleOnline);
    window.removeEventListener('offline', handleOffline);
  };
}, []);

// Flight Alerts Polling System
useEffect(() => {
  if (!trackedActivity) {
    setAlertMessage(undefined);
    return;
  }

  const pollFlight = async () => {
    try {
      const flights = await fetchRealFlights(trackedActivity.ident);
      if (flights && flights.length > 0) {
        const updatedFlight = flights[0];

        // Detect changes
        let alert = '';
        if (updatedFlight.status !== trackedActivity.status) {
          alert = `Status changed to ${updatedFlight.status}`;
        } else if (updatedFlight.gate && updatedFlight.gate !== trackedActivity.gate) {
          alert = `Gate changed to ${updatedFlight.gate}`;
        } else if (updatedFlight.delayMinutes && updatedFlight.delayMinutes > (trackedActivity.delayMinutes || 0)) {
          alert = `Delayed by ${updatedFlight.delayMinutes} mins`;
        }

        if (alert) {
          setAlertMessage(alert);
        }
        setTrackedActivity(updatedFlight);
      }
    } catch (e) {
      console.error("Flight poll failed", e);
    }
  };

  // Poll every 10 seconds for premium telemetry
```

```
    const timer = setInterval(pollFlight, 10000);
    return () => clearInterval(timer);
  }, [trackedActivity]);

// Handle Theme Change
const handleThemeChange = (newTheme: Theme) => {
  setTheme(newTheme);
  localStorage.setItem('urtc_theme', newTheme);
};

useEffect(() => {
  const root = document.documentElement;
  if (theme === 'light') root.classList.remove('dark'); else root.classList.add('dark');
}, [theme]);

// Handle Text Size Change (Global Scaling)
useEffect(() => {
  const root = document.documentElement;
  if (textSize === 'sm') root.style.fontSize = '14px';
  else if (textSize === 'lg') root.style.fontSize = '18px';
  else root.style.fontSize = '16px';
}, [textSize]);

// Load Saved Data
useEffect(() => {
  const savedBudget = localStorage.getItem('urtc_budget');
  if (savedBudget) setBudgetItems(JSON.parse(savedBudget));
  const savedLimit = localStorage.getItem('urtc_budget_limit');
  if (savedLimit) setBudgetLimit(Number(savedLimit));
  const savedNotes = localStorage.getItem('urtc_notes');
  if (savedNotes) setNotes(JSON.parse(savedNotes));
  const savedPasses = localStorage.getItem('urtc_passes');
  if (savedPasses) setPasses(JSON.parse(savedPasses));
}, []);

useEffect(() => { localStorage.setItem('urtc_budget', JSON.stringify(budgetItems)); }, [budgetItems]);
useEffect(() => { localStorage.setItem('urtc_budget_limit', budgetLimit.toString()); }, [budgetLimit]);
useEffect(() => { localStorage.setItem('urtc_notes', JSON.stringify(notes)); }, [notes]);
useEffect(() => { localStorage.setItem('urtc_passes', JSON.stringify(passes)); }, [passes]);

const [mountedTabs, setMountedTabs] = useState<Set<Tab>>>(new Set([Tab.Home]));

const [showApolloSheet, setShowApolloSheet] = useState(false);

const handleTabChange = React.useCallback((newTab: Tab) => {
  // Apollo is no longer a tab – it's the concierge sheet, available everywhere
  if (newTab === Tab.Apollo) {
    setMountedTabs(prev => { const s = new Set(prev); s.add(Tab.Apollo); return s; });
    setShowApolloSheet(true);
    return;
  }
  setMountedTabs(prev => { const s = new Set(prev); s.add(newTab); return s; });
  setActiveTab(newTab);
}, []);
```



```
const addToBudget = React.useCallback((item: BudgetItem) => {
  setBudgetItems(prev => [...prev, item]);
}, []);

const handleViewDestination = React.useCallback((city: string) => {
  setDirection(1);
  setExploreCity(city);
  setActiveTab(Tab.Explore);
  setMountedTabs(prev => { const s = new Set(prev); s.add(Tab.Explore); return s; });
}, []);

const handleTabSelect = React.useCallback((tab: Tab) => {
  handleTabChange(tab);
}, [handleTabChange]);

const handleBackToHome = React.useCallback(() => {
  handleTabChange(Tab.Home);
}, [handleTabChange]);

const handleExplore = React.useCallback((city: string) => {
  setExploreCity(city);
  handleTabChange(Tab.Explore);
}, [handleTabChange]);

const handleStartTour = React.useCallback(() => {
  setRunTour(true);
}, []);

const handlers = useSwipeable({
  onSwipedLeft: (e) => {
    if (Math.abs(e.deltaX) < 80 || Math.abs(e.deltaY) > Math.abs(e.deltaX) * 0.6) return;
    const currentIndex = TAB_ORDER.indexOf(activeTab);
    if (currentIndex < TAB_ORDER.length - 1) handleTabChange(TAB_ORDER[currentIndex + 1]);
  },
  onSwipedRight: (e) => {
    if (Math.abs(e.deltaX) < 80 || Math.abs(e.deltaY) > Math.abs(e.deltaX) * 0.6) return;
    const currentIndex = TAB_ORDER.indexOf(activeTab);
    if (currentIndex > 0) handleTabChange(TAB_ORDER[currentIndex - 1]);
  },
  preventScrollOnSwipe: false,
  trackMouse: false,
  delta: 60,
  swipeDuration: 500
});

// Tabs that should be full-screen (no header padding)
const immersiveTabs = [Tab.Wander, Tab.Flights, Tab.Explore];
const isImmersive = immersiveTabs.includes(activeTab);

return (
  <div className={theme}>
    {showSplash ? (
      <SplashScreen onComplete={() => setShowSplash(false)} />
    )
  </div>
);
```

```

) : (
  <div className={`min-h-screen ${
    theme === 'amoled' ? 'bg-black' :
    theme === 'light' ? 'bg-[#F2F4F7]' :
    'bg-brand-ink'
  } text-white selection:bg-brand-orange selection:text-white transition-colors duration-500`}>

    {/* Ambient depth glows removed for performance */}

    <div className="max-w-md sm:max-w-lg md:max-w-2xl lg:max-w-3xl xl:max-w-4xl mx-auto min-h-screen relative">

      {/* Flight tracking Dynamic Island */}
      {trackedActivity && (
        <div className="fixed top-safe left-4 right-4 z-50 max-w-md mx-auto animate-in fade-in slide-in-from-top-4 duration-300">
          <DynamicIsland activity={trackedActivity} alertMessage={alertMessage} onClose={() => setTrackedActivity(null)} />
        </div>
      )}

      {showDiamondTutorial && <DiamondTutorialOverlay onClose={handleCloseDiamondTutorial} />}
      {runTour && <TutorialOverlay onClose={() => setRunTour(false)} />}

      {/* Offline Banner */}
      {isOffline && (
        <div className="fixed top-0 left-0 w-full z-[60] bg-red-500/90 backdrop-blur-md text-white text-xs font-bold text-center pb-2 pt-safe-header flex justify-center gap-2 items-center">
          <WifiOff size={12} /> No Internet Connection
        </div>
      )}

      {/* ■■■ Floating Glass Header (non-immersive tabs only) ■■■ */}
      {!isImmersive && (
        <header className="sticky top-0 z-30 px-4 pt-safe-header pb-3">
          <div className="glass rounded-2xl px-4 py-2.5 flex items-center justify-between">
            {/* Logo */}
            <div className="flex items-center gap-2.5">
              <div className="relative">
                <div className="absolute inset-0 bg-brand-orange/30 blur-sm rounded-lg" />
                <div className="relative bg-gradient-to-br from-brand-orange to-orange-700 w-8 h-8 rounded-xl flex items-center justify-center shadow-lg">
                  <Plane size={15} className="text-white" strokeWidth={2.5} />
                </div>
              </div>
              <div>
                <div className="flex items-center gap-2">
                  <h1 className="font-display text-lg font-bold tracking-tight leading-none text-white">
                    Ü<span className="gradient-text">TC</span>
                  </h1>
                  <span className="text-[9px] font-bold bg-brand-orange/20 text-brand-orange px-1.5 py-0.5 rounded-full">v1.1</span>
                </div>

```

```

        <div className="flex items-center gap-1 mt-0.5">
          <span className={`w-1.5 h-1.5 rounded-full ${
            user.tier === UserTier.Diamond || user.tier === UserTier.Professional || user.t
            ier === UserTier.Dev
              ? 'bg-green-400 animate-pulse' : 'bg-white/20'
            }`} />
          <p className="text-[9px] text-white/30 font-medium uppercase tracking-wider">
            {user.tier === UserTier.Dev ? 'Dev' :
              user.tier === UserTier.Diamond ? 'Diamond' :
              user.tier === UserTier.Professional ? 'Professional' :
              user.tier === UserTier.Free ? 'Silver' : 'Bronze'}
          </p>
        </div>
      </div>
    </div>

    {/* Right controls */}
    <div className="flex items-center gap-1.5">
      <button
        onClick={() => handleTabChange(Tab.About)}
        className="p-2 text-white/40 hover:text-white rounded-xl hover:bg-white/5 transitio
n"
        title="Settings & plans"
      >
        <Settings size={17} />
      </button>
      <button
        onClick={() => handleThemeChange(theme === 'dark' ? 'light' : 'dark')}
        className="p-2 text-white/40 hover:text-white rounded-xl hover:bg-white/5 transitio
n"
      >
        {theme === 'dark' ? <Moon size={17} /> : <Sun size={17} />}
      </button>
      <button
        onClick={() => setShowLive(true)}
        className="w-8 h-8 flex items-center justify-center rounded-xl bg-brand-orange/15 b
order border-brand-orange/25 text-brand-orange hover:bg-brand-orange hover:text-white transition"
      >
        <Mic size={15} />
      </button>
    </div>
  </div>

</header>
)}

{/* ■■■ Main Content (lazy-mount: only initialize a tab when first visited, then keep alive)
■■■ */}
<main className="relative z-10 pb-32">
  {locationDenied && (
    <div className="mx-4 mt-2 mb-2 p-3 bg-red-500/10 border border-red-500/20 rounded-xl flex
items-start gap-3 text-red-500 dark:text-red-400 animate-in slide-in-from-top-4">
      <MapPinOff size={20} className="shrink-0 mt-0.5" />
      <div className="text-xs leading-relaxed">

```

```

        <strong className="block font-bold mb-1">Location Services Disabled</strong>
        ÜrTC requires location access to auto-detect your city for live weather and map data.
        Please enable location permissions in your browser/device settings.
    </div>
</div>
))
<div style={{ display: activeTab === Tab.Home ? 'block' : 'none' }} className="px-4 pt-3">
    {activeTab === Tab.Home && <HomeView user={user} onNavigate={handleTabChange} onExplore={
handleExplore} onStartTour={handleStartTour} budgetItems={budgetItems} budgetLimit={budgetLimit} />}
    </div>
    {mountedTabs.has(Tab.Flights) && (
        <div className="pt-safe-top" style={{ display: activeTab === Tab.Flights ? 'block' : 'non
e' }}>
            <FlightView user={user} onViewCity={handleViewDestination} onTrackFlight={setTrackedAct
ivity} />
        </div>
    )}
    {mountedTabs.has(Tab.Explore) && (
        <div className="pt-safe-top" style={{ display: activeTab === Tab.Explore ? 'block' : 'non
e' }}>
            <CityView onAddToBudget={addToBudget} initialCity={exploreCity} onCityChange={setExplor
eCity} theme={theme} />
        </div>
    )}
    {mountedTabs.has(Tab.Wander) && (
        <div className="pt-safe-top" style={{ display: activeTab === Tab.Wander ? 'block' : 'none
' }}>
            <SocialView />
        </div>
    )}
    {mountedTabs.has(Tab.Itinerary) && (
        <div style={{ display: activeTab === Tab.Itinerary ? 'block' : 'none' }} className="px-4
pt-3">
            <ItineraryView user={user} onAskApollo={() => handleTabChange(Tab.Apollo)} />
        </div>
    )}
    {mountedTabs.has(Tab.About) && (
        <div style={{ display: activeTab === Tab.About ? 'block' : 'none' }} className="px-4 pt-3
">
            <AboutView currentUser={user} onUserUpdate={setUser} textSize={textSize} onTextSizeChan
ge={setTextSize} />
        </div>
    )}
</main>

<div className="fixed bottom-0 left-0 right-0 z-40 pb-safe pb-4 pt-2 px-4 md:px-8 lg:px-16">
    <div className="pill-nav rounded-[28px] px-2 py-2 flex items-center justify-between relativ
e shadow-2xl shadow-black/50 w-full max-w-md sm:max-w-lg md:max-w-2xl mx-auto">

        {/* Apollo FAB (elevated center) */}
        <div className="absolute left-1/2 -translate-x-1/2 -top-6 flex flex-col items-center">
            <button
                id="tab-apollo"
                onClick={() => handleTabChange(Tab.Apollo)}

```

```

        className={`relative select-none ${showApolloSheet ? 'animate-glow-pulse' : ''}`}
      >
      <div className={`absolute inset-0 rounded-full blur-md transition-opacity ${
        showApolloSheet ? 'bg-brand-orange/60 opacity-100' : 'opacity-0'
      }`} />
      <div className={`relative w-14 h-14 rounded-full border-[3px] ${
        showApolloSheet ? 'border-brand-orange shadow-lg glow-orange' : 'border-brand-orang
e/40'

      } overflow-hidden bg-brand-surface transition-all duration-200`}>
        
      </div>
      <span className="absolute -top-0.5 -right-0.5 w-3 h-3 bg-green-400 border-2 border-br
and-surface rounded-full" />
    </button>
    <span className={`text-[9px] font-bold uppercase tracking-wider mt-1 ${
      showApolloSheet ? 'text-brand-orange' : 'text-white/25'
    }`}>Apollo</span>
  </div>

  {/* Left tabs */}
  {[
    { tab: Tab.Home, icon: <Home size={22} />, label: 'Today' },
    { tab: Tab.Flights, icon: <Plane size={22} />, label: 'Flights' },
  ].map(({ tab, icon, label }) => (
    <button
      key={tab}
      id={`tab-${tab}`}
      onClick={() => handleTabChange(tab)}
      className={`flex flex-col items-center justify-center gap-1 flex-1 py-2 rounded-2xl t
ransition-colors duration-150 relative select-none ${
        activeTab === tab ? 'text-brand-orange' : 'text-white/40'
      }`}
    >
      {activeTab === tab && (
        <div className="absolute inset-0 bg-brand-orange/10 rounded-2xl pointer-events-none
" />

        )}
        <div className="relative">{icon}</div>
        <span className="text-[9px] font-bold uppercase tracking-wide">{label}</span>
      </button>
    )]}

  {/* Spacer for Apollo */}
  <div className="flex-1 shrink-0" />

  {/* Right tabs */}
  {[
    { tab: Tab.Explore, icon: <Building2 size={22} />, label: 'Explore' },
    { tab: Tab.Itinerary, icon: <Notebook size={22} />, label: 'Trips' },
  ].map(({ tab, icon, label }) => (

```

```

        <button
          key={tab}
          id={`tab-${tab}`}
          onClick={() => handleTabChange(tab)}
          className={`flex flex-col items-center justify-center gap-1 flex-1 py-2 rounded-2xl t
ransition-colors duration-150 relative select-none ${
            activeTab === tab ? 'text-brand-orange' : 'text-white/40'
          }}`
        >
          {activeTab === tab && (
            <div className="absolute inset-0 bg-brand-orange/10 rounded-2xl pointer-events-none
" />

            )}
            <div className="relative">{icon}</div>
            <span className="text-[9px] font-bold uppercase tracking-wide">{label}</span>
          </button>
        )}}
      </div>
    </div>

    {/ * ■■ Apollo Concierge Sheet – slides over any screen ■■ */}
    {mountedTabs.has(Tab.Apollo) && (
      <div className={`fixed inset-0 z-[90] ${showApolloSheet ? '' : 'pointer-events-none'}}`>
        <div
          className={`absolute inset-0 bg-black/60 backdrop-blur-sm transition-opacity duration-3
00 ${showApolloSheet ? 'opacity-100' : 'opacity-0'}}`
          onClick={() => setShowApolloSheet(false)}
        />
        <div className={`absolute bottom-0 left-1/2 -translate-x-1/2 w-full max-w-md sm:max-w-lg
md:max-w-2xl bg-[#0f1115] border border-white/10 rounded-t-[32px] shadow-2xl h-[88vh] flex flex-col ove
rflow-hidden transition-transform duration-300 ${showApolloSheet ? 'translate-y-0' : 'translate-y-full'
}`}>
          <div className="flex items-center justify-between px-5 pt-4 pb-3 border-b border-white/
5 shrink-0">
            <div className="flex items-center gap-2.5">
              <div className="relative">
                <div className="w-9 h-9 rounded-full overflow-hidden border-2 border-brand-orange
/50">
                  
                </div>
                <span className="absolute -bottom-0.5 -right-0.5 w-3 h-3 bg-green-400 border-2 bo
rder-[#0f1115] rounded-full" />
              </div>
              <div>
                <div className="text-sm font-black text-white leading-none">Apollo</div>
                <div className="text-[9px] text-white/40 uppercase tracking-widest mt-1">Your tra
vel companion</div>
              </div>
            </div>
            <button onClick={() => setShowApolloSheet(false)} className="p-2 text-white/40 hover:
text-white rounded-full hover:bg-white/10 transition">
              <X size={18} />
            </button>

```

```

        </div>
        <div className="flex-1 overflow-y-auto px-4 pt-2 pb-4">
            <ApolloView userTier={user.tier} onBack={() => setShowApolloSheet(false)} />
        </div>
    </div>
</div>
)}}

{showLive && <ApolloLive isOpen={showLive} onClose={() => setShowLive(false)} />}
</div>

{/* Terms & Agreements Modal */}
{showTerms && (
    <div className="fixed inset-0 z-[200] flex items-center justify-center bg-black/90 backdrop-blur-md p-4">
        <div className="w-full max-w-lg bg-[#0f1115] border border-white/20 rounded-3xl overflow-hidden shadow-2xl flex flex-col max-h-[90vh]">
            <div className="p-6 border-b border-white/10 bg-[#151921]">
                <div className="flex items-center gap-3 mb-2">
                    <div className="bg-brand-orange/20 p-2 rounded-xl"><Shield size={24} className="text-brand-orange" /></div>
                    <div>
                        <h2 className="text-2xl font-black text-white">Terms & Agreements</h2>
                        <p className="text-xs text-gray-500 font-bold uppercase tracking-widest">ÜrTC – Cave Core Dynamics™</p>
                    </div>
                </div>
            </div>
            <div>
                <div className="flex-1 overflow-y-auto p-6 text-sm text-gray-300 leading-relaxed space-y-5 scrollbar-hide">
                    <div className="space-y-3">
                        <h3 className="text-white font-bold text-base">1. Acceptance of Terms</h3>
                        <p>By accessing or using ÜrTC ("the App"), you acknowledge that you have read, understood, and agree to be bound by these Terms & Conditions. If you do not agree, you may not use the App.</p>
                    </div>

                    <div className="space-y-3">
                        <h3 className="text-white font-bold text-base">2. AI-Generated Content Disclaimer</h3>
                        <p>This application uses <strong className="text-brand-orange">Google Gemini AI</strong> ("Apollo") to generate travel insights, budget recommendations, and conversational responses. <strong>AI can make mistakes.</strong> All AI-generated content is provided for informational purposes only and should not be considered professional travel, financial, or legal advice. Always verify critical information independently.</p>
                    </div>

                    <div className="space-y-3">
                        <h3 className="text-white font-bold text-base">3. Third-Party Data & APIs</h3>
                        <p>Flight data is sourced from <strong className="text-brand-orange">FlightAware Aero API</strong>, weather from <strong>OpenWeather</strong>, and maps from <strong>Google Maps Platform</strong>. Data accuracy, availability, and pricing are controlled by these third-party providers and may change at any time without notice. Cave Core Dynamics™ is not responsible for inaccuracies in third-party data.</p>
                    </div>
                </div>
            </div>
        </div>
    )}

```

```
</div>

<div className="space-y-3">
  <h3 className="text-white font-bold text-base">4. No Warranty & Limitation of Liability</h3>

  <p>The App is provided <strong>"AS IS"</strong> without warranty of any kind. Cave Core Dynamics™, its founders, employees, and affiliates shall not be held liable for any direct, indirect, incidental, or consequential damages arising from your use of the App, including but not limited to missed flights, financial losses, or reliance on AI-generated content.</p>
</div>

<div className="space-y-3">
  <h3 className="text-white font-bold text-base">5. Pricing & Subscriptions</h3>
  <p>Subscription pricing (Diamond, Professional) is subject to change. Current prices are displayed at the time of purchase. By subscribing, you agree to the pricing shown at checkout. Refund policies are governed by the payment processor (Stripe).</p>
</div>

<div className="space-y-3">
  <h3 className="text-white font-bold text-base">6. Cookies & Local Storage</h3>
  <p>By using this App, you consent to the use of <strong>cookies and browser local storage</strong> to store your preferences, session data, trip information, and authentication tokens. This data is stored locally on your device and is essential for the App to function properly.</p>
</div>

<div className="space-y-3">
  <h3 className="text-white font-bold text-base">7. Privacy & Data</h3>
  <p>We collect only the data necessary to provide the App's services. Account data is stored securely via Google Cloud (Firebase). We do not sell your personal information to third parties. For full details, contact <a href="mailto:feedback@cavecoredynamics.org" className="text-brand-orange hover:underline">feedback@cavecoredynamics.org</a>.</p>
</div>

<div className="space-y-3">
  <h3 className="text-white font-bold text-base">8. Dispute Resolution</h3>
  <p>Any disputes arising from your use of this App shall be resolved through binding arbitration in the State of Georgia, United States, in accordance with applicable laws. By using this App, you waive your right to participate in class-action lawsuits.</p>
</div>

<div className="bg-brand-orange/10 border border-brand-orange/30 rounded-xl p-4 text-xs text-brand-orange">
  <p className="font-bold mb-1">■■ Important Reminder</p>
  <p>Gemini AI (Apollo) can make mistakes. Flight data from APIs may be delayed or inaccurate. Pricing for third-party services and subscriptions may change over time. Always double-check critical travel information.</p>
</div>

<p className="text-[10px] text-gray-500 pt-2">Last updated: June 2026 — © Cave Core Dynamics™. All rights reserved.</p>
</div>
<div className="p-4 bg-[#151921] border-t border-white/10">
  <button
    onClick={() => { localStorage.setItem('urtc_terms_accepted', 'true'); setShowTerms(false); }}
  >
```



```

lse); }}

        className="w-full py-3.5 bg-gradient-to-r from-brand-orange to-red-500 text-white rounded-xl font-bold text-sm shadow-lg shadow-brand-orange/20 hover:scale-[1.02] active:scale-[0.97] transition-all"
        >
            I Accept – Continue to ÜrTC
        </button>
        <p className="text-center text-[10px] text-gray-500 mt-2">By clicking "I Accept" you agree to all terms above including cookies usage.</p>
    </div>
</div>
</div>
    )}
</div>
    )}
</div>
);
};

```

```
export default AppContent;
```

```
=====
FILE: config.ts
=====
```

```

/// <reference types="vite/client" />
// API keys live in .env.local (git-ignored). See .env.example for the shape.
export const API_KEYS = {
    FLIGHTAWARE: import.meta.env.VITE_FLIGHTAWARE_API_KEY || "",
    OWM: import.meta.env.VITE_OWM_API_KEY || "",
    GOOGLE_MAPS: import.meta.env.VITE_GOOGLE_MAPS_API_KEY || "",
    GEMINI: import.meta.env.VITE_GEMINI_API_KEY
};

```

```
=====
FILE: types.ts
=====
```

```

export enum FlightStatus {
    Scheduled = "Scheduled",
    OnTime = "On Time",
    Delayed = "Delayed",
    Cancelled = "Cancelled",
    Boarding = "Boarding",
    EnRoute = "En Route",
    Landed = "Landed"
}

export interface DelayInsight {
    reason: string;
    confidence: number;
    description: string;
}

```

```
export interface FlightPosition {
  latitude: number;
  longitude: number;
  altitude: number;
  timestamp: string;
  groundspeed?: number;
  heading?: number;
}
```

```
export interface Flight {
  id: string;
  ident: string;
  flightNumber: string;
  airline: string;
  status: FlightStatus;
  departureAirport: string;
  arrivalAirport: string;
  departureTime: string;
  arrivalTime: string;
  gate?: string;
  terminal?: string;
  gateDestination?: string;
  terminalDestination?: string;
  baggage?: string;
  estimatedDepartureTime?: string;
  actualDepartureTime?: string;
  estimatedArrivalTime?: string;
  actualArrivalTime?: string;
  aircraft?: string;
  tailNumber?: string;
  progress: number;
  delayMinutes?: number;
  durationMinutes?: number | null;
  price?: number;
  latitude?: number;
  longitude?: number;
  heading?: number;
  altitude?: number;
  insight?: DelayInsight;
  websiteUrl?: string;
  inboundFlight?: {
    flightNumber: string;
    origin: string;
    status: string;
    arrivalTime: string;
  };
  track?: FlightPosition[];
}
```

```
export interface Airport {
  code: string;
  name: string;
  city: string;
```

```
    state: string;
    tsaWaitMinutes: number;
    isPreCheckOpen: boolean;
    lat?: number;
    lng?: number;
  }

export interface HourlyForecast {
  time: string;
  temp: number;
  condition: string;
  icon?: string;
}

export interface DailyForecast {
  date: string;
  minTemp: number;
  maxTemp: number;
  condition: string;
  icon?: string;
}

export interface Weather {
  temp: number;
  condition: string;
  humidity: number;
  wind: number;
  city: string;
  feelsLike?: number;
  hourly?: HourlyForecast[];
  daily?: DailyForecast[];
}

export interface Place {
  id: string;
  name: string;
  category: 'Hotel' | 'Food' | 'Attraction';
  rating: number;
  priceLevel: number;
  image: string;
  priceEstimate: number;
  priceDisplay: string;
  coordinates: { x: number; y: number };
  description: string;
  websiteUrl?: string;
  distanceText?: string;
  durationText?: string;
}

export interface NewsArticle {
  id: string;
  title: string;
  source: string;
  url: string;
```

```
    publishedAt: string;
    imageUrl?: string;
  }

export interface ChatMessage {
  id: string;
  text: string;
  sender: 'user' | 'apollo';
  timestamp: Date;
}

export enum Tab {
  Home = 'home',
  Fun = 'fun',
  Flights = 'flights',
  Explore = 'explore',
  Wander = 'wander', // Social Tab
  Apollo = 'apollo', // Center Tab (AI)
  Itinerary = 'itinerary', // Plans (Itinerary + Docs + Auth)
  About = 'about'
}

export interface BudgetItem {
  id: string;
  name: string;
  cost: number;
  category: 'Flight' | 'Food' | 'Attraction' | 'Hotel' | 'Other';
}

export interface Note {
  id: string;
  tripName: string;
  city: string;
  stateCountry: string;
  title: string;
  content: string;
  date: Date;
  isAiGenerated?: boolean;
}

export interface TripFlight {
  id: string;
  trip_id: string;
  flight_number: string;
  airline?: string;
  flight_date: string;
  departure_airport?: string;
  arrival_airport?: string;
  status: string;
}

export interface BudgetCategory {
  id: string;
  type: 'Flight' | 'Hotel' | 'Food' | 'Attraction' | 'Bars & Nightlife' | 'Other';
}
```

```
    label: string;
    planned: number;
    allocated?: number;
  }

export interface Trip {
  id: string;
  user_id: string;
  name: string;
  start_date?: string;
  end_date?: string;
  created_at: string;
  archived?: boolean;
  budget_limit?: number;
  destination?: string;
  travelers_count?: number;
  duration_days?: number;
  notes: Note[];
  budget_categories: BudgetCategory[];
  passes: Pass[];
  flights?: TripFlight[]; // Joined dynamically
  places?: Place[]; // Places saved from Explore
}

export interface Pass {
  id: string;
  type: 'Boarding Pass' | 'Hotel' | 'Train' | 'Event' | 'Other';
  provider: string;
  reference: string;
  details: string;
  date?: string;
}

export enum UserTier {
  Guest = "Bronze", // Guest
  Free = "Silver",   // Free
  Diamond = "Diamond", // Pro
  Professional = "Professional", // Crew
  Dev = "Dev"        // Dev
}

export type Theme = 'light' | 'dark' | 'amoled';
export type Language = 'en' | 'es' | 'fr' | 'de';

export interface UserPreferences {
  textSize: 'sm' | 'base' | 'lg';
  language: Language;
}

export interface GameState {
  isPlaying: boolean;
  score: number;
  highScore: number;
}
```

```
export interface FlightSchedule {
  ident: string;
  departure: string;
  arrival: string;
  origin: string;
  destination: string;
  airline?: string;
  aircraft?: string;
}

export interface AirportConditions {
  airportCode: string;
  delayIndex: number;
  delayReasons?: string[];
  weather: string;
  temp: number;
  visibility: string;
  wind: string;
}

export interface GateInfo {
  gate: string;
  status: string;
  flight: string | null;
}

export interface AustralianAirspace {
  activeFlights: number;
  restrictions: string[];
  weatherAlerts: string[];
}

export interface UserAccount {
  id: string;
  username: string;
  email?: string;
  passwordHash: string;
  tier: UserTier;
  trialEndsAt?: string; // ISO date – free Diamond trial expiry
  savedTrips: string[]; // IDs of saved trips
  promoOptIn?: boolean;
  rememberMe?: boolean;
  avatarUrl?: string;
  bio?: string;
  isPrivate?: boolean;
}

// === AeroAPI Premium Types ===

export interface ForesightPrediction {
  predicted_out?: string;
  predicted_off?: string;
  predicted_on?: string;
}
```

```
    predicted_in?: string;
    predicted_out_source?: string;
    predicted_off_source?: string;
    predicted_on_source?: string;
    predicted_in_source?: string;
    predicted_taxi_out_duration?: number;
    predicted_taxi_out_duration_source?: string;
  }
```

```
export interface FlightRouteWaypoint {
  name: string;
  latitude: number;
  longitude: number;
  distance_from_origin: number;
  distance_this_leg: number;
  distance_to_destination: number;
  outbound_course: number;
  type: string;
}
```

```
export interface FlightRoute {
  route_distance: string;
  fixes: FlightRouteWaypoint[];
}
```

```
export interface AirportInfo {
  code: string;
  code_icao: string;
  code_iata: string;
  name: string;
  city: string;
  state?: string;
  country?: string;
  timezone: string;
  latitude: number;
  longitude: number;
  wiki_url?: string;
  airport_info_url?: string;
}
```

```
export interface AirportFlightCounts {
  departed: number;
  enroute: number;
  scheduled_departures: number;
  scheduled_arrivals: number;
}
```

```
export interface AirportWeatherForecast {
  conditions: string;
  temp_air: number;
  temp_dewpoint: number;
  temp_perceived: number;
  wind_speed: number;
  wind_direction: number;
}
```

```
visibility: string;
cloud_coverage: string;
pressure: number;
timestamp: string;
}

export interface NearbyAirport {
  code: string;
  name: string;
  city: string;
  distance: number;
  heading: number;
}

export interface RouteStats {
  route_distance: number;
  avg_duration: number;
  filed_altitude: number;
  last_departure_time: string;
}

export interface OperatorInfo {
  icao: string;
  iata: string;
  name: string;
  callsign: string;
  country: string;
  location: string;
  shortname: string;
}

export interface OperatorFlightCounts {
  airborne: number;
  flights_last_24_hours: number;
}

export interface AircraftOwner {
  name: string;
  location: string;
  location2?: string;
  state?: string;
  website?: string;
}

export interface AircraftTypeInfo {
  type: string;
  manufacturer: string;
  description: string;
  engine_type?: string;
  engine_count?: number;
}

export interface DisruptionCount {
  entity_type: string;
```



```

    entity_id?: string;
    delays: number;
    cancellations: number;
    total: number;
}

export interface FlightAlert {
    id: number;
    ident?: string;
    origin?: string;
    destination?: string;
    aircraft_type?: string;
    date_start?: string;
    date_end?: string;
    channels: { target_url: string; channel_type: string }[];
    events: string[];
    enabled: boolean;
}

export interface GlobalDelay {
    airport_code: string;
    delay_index: number;
    reasons: string[];
}

```

```

=====
FILE: index.html
=====

```

```

<!DOCTYPE html>
<html lang="en" class="dark">

<head>
  <meta charset="UTF-8" />
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=0, viewport-fit=cover" />
  <title>ÜrTC - Travel Companion</title>
  <meta name="description" content="ÜrTC by Cave Core Dynamics – Your AI-powered travel companion. Track flights, explore cities, plan budgets, and get help from Apollo AI. Travel Evolved." />
  <meta name="theme-color" content="#08090C" />
  <meta name="apple-mobile-web-app-capable" content="yes" />
  <meta name="apple-mobile-web-app-status-bar-style" content="black-translucent" />
  <meta name="apple-mobile-web-app-title" content="ÜrTC" />
  <link rel="manifest" href="/manifest.webmanifest" />
  <meta name="theme-color" content="#0B0D10" />
  <meta name="mobile-web-app-capable" content="yes" />
  <meta name="application-name" content="ÜrTC" />
  <meta property="og:title" content="ÜrTC - Travel Evolved" />
  <meta property="og:description" content="AI-powered travel companion by Cave Core Dynamics. Track flights via FlightAware, explore cities, plan trips with Apollo AI." />
  <meta property="og:type" content="website" />
  <meta property="og:url" content="https://cavecoredynamics.org/urtc" />
  <link rel="canonical" href="https://cavecoredynamics.org/urtc" />

```

```
<!-- Premium Google Fonts: Space Grotesk + Inter -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@300;400;500;600;700&family=Inter:wght@300;400;500;600;700;800&display=swap" rel="stylesheet">

<!-- Tailwind CDN -->
<script src="https://cdn.tailwindcss.com"></script>

<!-- Google Maps – v=beta required for the photorealistic 3D (maps3d) flight map -->
<script src="https://maps.googleapis.com/maps/api/js?key=%VITE_GOOGLE_MAPS_API_KEY%&libraries=places,marker,geometry&v=beta"></script>

<!-- Google AdSense -->
<script async src="https://pagead2.googlesyndication.com/pagead/js/adsbygoogle.js?client=ca-pub-9455431514273237" crossorigin="anonymous"></script>

<!-- Viator Banner Ads -->
<script async src="https://partners.vtrcdn.com/static/scripts/banners/banners.js"></script>

<!-- Stay22 Affiliate Script -->
<script>
  (function (s, t, a, y, twenty, two) {
    s.Stay22 = s.Stay22 || {};
    s.Stay22.params = { lmaID: '6a3e5baf1b730bc0adfe1f08' };
    twenty = t.createElement(a);
    two = t.getElementsByTagName(a)[0];
    twenty.async = 1;
    twenty.src = y;
    two.parentNode.insertBefore(twenty, two);
  })(window, document, 'script', 'https://scripts.stay22.com/letmeallez.js');
</script>

<script>
  tailwind.config = {
    darkMode: 'class',
    theme: {
      extend: {
        fontFamily: {
          sans: ['Inter', 'sans-serif'],
          display: ['Space Grotesk', 'sans-serif'],
        },
        colors: {
          brand: {
            orange: '#FF5C1A',
            'orange-glow': 'rgba(255,92,26,0.15)',
            blue: '#2E86FF',
            'blue-glow': 'rgba(46,134,255,0.15)',
            dark: '#08090C',
            ink: '#08090C',
            surface: '#10131A',
            elevated: '#161B25',
            glass: 'rgba(255,255,255,0.04)',
            border: 'rgba(255,255,255,0.07)',
          },
        },
      },
    },
  };
</script>
```

```
    apollo: '#FFB800',
    'apollo-glow': 'rgba(255,184,0,0.15)',
  }
},
animation: {
  'float': 'float 6s ease-in-out infinite',
  'fly-straight': 'flyStraight 2.5s ease-in-out forwards',
  'fade-in-text': 'fadeInText 2.5s ease-in-out forwards',
  'smoke-reveal': 'smokeReveal 2s ease-out forwards',
  'pulse-slow': 'pulse 3s cubic-bezier(0.4, 0, 0.6, 1) infinite',
  'shimmer': 'shimmer 2s linear infinite',
  'wag': 'wag 0.5s ease-in-out infinite alternate',
  'slide-up': 'slideUp 0.5s cubic-bezier(0.16,1,0.3,1) forwards',
  'glow-pulse': 'glowPulse 2.5s ease-in-out infinite',
  'blur-in': 'blurIn 0.4s ease-out forwards',
  'scale-in': 'scaleIn 0.35s cubic-bezier(0.16,1,0.3,1) forwards',
},
keyframes: {
  float: {
    '0%, 100%': { transform: 'translateY(0)' },
    '50%': { transform: 'translateY(-10px)' },
  },
  flyStraight: {
    '0%': { transform: 'translateX(-120vw)', opacity: '0' },
    '10%': { opacity: '1' },
    '100%': { transform: 'translateX(120vw)', opacity: '1' }
  },
  fadeInText: {
    '0%': { opacity: '0', transform: 'translateX(-50px)' },
    '40%': { opacity: '0' },
    '60%': { opacity: '1', transform: 'translateX(0)' },
    '100%': { opacity: '0', transform: 'translateX(50px)' }
  },
  smokeReveal: {
    '0%': { opacity: '0', transform: 'scale(0.8) translateY(20px)', filter: 'blur(20px)' },
    '60%': { opacity: '0.8', filter: 'blur(5px)' },
    '100%': { opacity: '1', transform: 'scale(1) translateY(0)', filter: 'blur(0px)' }
  },
  shimmer: {
    '0%': { transform: 'translateX(-100%)' },
    '100%': { transform: 'translateX(100%)' }
  },
  wag: {
    '0%': { transform: 'rotate(-10deg)' },
    '100%': { transform: 'rotate(10deg)' }
  },
  slideUp: {
    '0%': { opacity: '0', transform: 'translateY(24px)' },
    '100%': { opacity: '1', transform: 'translateY(0)' }
  },
  glowPulse: {
    '0%, 100%': { boxShadow: '0 0 20px rgba(255,92,26,0.3), 0 0 40px rgba(255,92,26,0.1)' },
    '50%': { boxShadow: '0 0 30px rgba(255,92,26,0.6), 0 0 60px rgba(255,92,26,0.2)' }
  },
},
```

LAST 25 PAGES (pages 484-508)

```

        return { error: e?.message || 'Tool execution failed' };
    }
};

const APOLLO_SYSTEM_PROMPT = (tripsContext: string) => {
    const lastCity = localStorage.getItem('urtc_last_city') || 'unknown';
    let flightCtx = 'none';
    try {
        const raw = localStorage.getItem('urtc_last_flight_context');
        if (raw) {
            const fc = JSON.parse(raw);
            // Only treat it as current if viewed within the last 48h
            if (fc.viewed_at && Date.now() - new Date(fc.viewed_at).getTime() < 48 * 60 * 60 * 1000) {
                flightCtx = JSON.stringify(fc);
            }
        }
    } catch (e) { /* ignore */ }
    const today = new Date().toLocaleDateString([], { weekday: 'long', year: 'numeric', month: 'long', day: 'numeric' });
    return `You are Apollo, the ÜrTC travel companion – a smart, loyal dog who happens to be a world-class travel expert.`;
};

```

PERSONALITY:

- Warm, sharp, and genuinely useful. You get people the right answer fast.
- Dog mannerisms: at most one subtle touch per response ("pawsome", a single "woof"). Never more. No asterisk actions.
- Keep responses short. Lead with the answer, then supporting detail.
- PLAIN TEXT ONLY: no markdown, no asterisks, no **bold**, no bullet symbols – the chat renders raw text. Use short lines and emoji sparingly for structure.
- Convert times to the local time of the relevant airport/city and say the timezone (e.g. "4:16 PM ET"). Never show raw UTC.

TRAVEL EXPERTISE:

- Give specific, actionable advice: real neighborhoods, dishes, transit lines, timing tips.
- For prices, give realistic current ranges and say they're estimates. Never invent exact prices for specific businesses.
- Flag safety/scam/weather considerations when genuinely relevant.
- If you don't know something current (e.g., today's flight status), say so and point to the app's Flights tab (live real-time data) instead of guessing. Never mention FlightAware or any external flight-tracking website – ÜrTC's Flights tab IS the flight tracker.

APP KNOWLEDGE (ÜrTC):

- Flights tab: live flight tracking, airport boards, delays.
- Explore tab: nearby food & attractions with prices and weather.
- Plans tab: trips, itineraries, budgets.
- Tiers: Free (basic tracking, manual budgets), Pro (AI predictions, live voice, smart budgets), Crew (dev access).

CONTEXT:

- Today: \${today}
- User's last browsed city: \${lastCity}
- Flight the user most recently viewed (treat as "my flight" when they say that): \${flightCtx}

YOUR HANDS (TOOLS) – you can actually DO things, not just talk:

- get_flight_status: ALWAYS use this when the user mentions a flight number – answer from real data, never guess.
- create_trip / add_flight_to_trip / list_trips: when the user asks you to plan, create, save, or track something – DO IT, then confirm in one friendly line what you did. Never say you can't create trips.
- Typical flow: user says "add DL1182 to my Tokyo trip" → get_flight_status → add_flight_to_trip → confirm with the flight's real status.

COMPANION BEHAVIOR:

- When the user asks about "my flight", their trip, delays, or timing, USE the flight context above – answer with its actual status, gate, and times instead of asking which flight.
 - If their flight is delayed, be proactive: mention the new time, and suggest what the delay means for connections, food time, or ground plans.
 - Connect the dots across the app: if they ask what to do near the airport during a delay, use their last browsed city and suggest the Explore tab.
 - `${tripsContext}`;`
- ```
};
```

```
export const streamApolloResponse = async (
 userMessage: string,
 history: { role: string; parts: { text: string }[] }[],
 onChunk: (text: string) => void
) => {
 const client = getClient();
 if (!client) {
 onChunk("Woof! My connection's a bit ruff (API Key missing).");
 return;
 }

 let tripsContext = "The user has no trips currently planned.";
 try {
 const user = getActiveUser();
 if (user && user.id) {
 const trips = await fetchTrips(user.id);
 if (trips && trips.length > 0) {
 tripsContext = `The user has ${trips.length} trips. Context: ` + JSON.stringify(trips.map(
 (t => ({
 name: t.name,
 flights: (t.flights || []).map((f: any) => `${f.airline} ${f.flight_number} to ${f.arrival_airport}`),
 budget: (t.budget_categories || []).reduce((a: number, c: any) => a + (c.planned || 0)),
 })))
);
 }
 }
 } catch (e) { console.error("Failed to load trips context", e); }

 // Clean the history: drop empty turns, cap at the last 20 to stay under token limits
 const contents = [
 ...history
 .filter(h => h.parts?.[0]?.text?.trim())
 .slice(-20)
 .map(h => ({ role: h.role === 'model' ? 'model' : 'user', parts: h.parts })),
 { role: 'user', parts: [{ text: userMessage }] }
];
};
```

```

const systemInstruction = APOLLO_SYSTEM_PROMPT(tripsContext);

// Round 1: agent mode with tools – Apollo can look up flights and edit trips.
// (Google Search grounding can't be combined with function tools, so tool mode
// runs first and we fall back to search-grounded chat if it fails.)
try {
 const user = getActiveUser();
 const userId = user?.id || 'guest';
 const convo: any[] = [...contents];
 for (let turn = 0; turn < 5; turn++) {
 const resp: any = await client.models.generateContent({
 model: 'gemini-2.5-flash',
 contents: convo,
 config: { systemInstruction, temperature: 0.7, tools: APOLLO_TOOLS }
 });
 const cand = resp.candidates?.[0];
 const parts = cand?.content?.parts || [];
 const calls = parts.filter((p: any) => p.functionCall);
 if (calls.length > 0) {
 convo.push(cand.content);
 for (const c of calls) {
 const result = await executeApolloTool(c.functionCall.name, c.functionCall.args || {}
, userId);
 convo.push({ role: 'user', parts: [{ functionResponse: { name: c.functionCall.name, r
esponse: { result } } }] });
 }
 continue; // let him read the tool results and keep working
 }
 const text = resp.text;
 if (text) { onChunk(text); return; }
 break;
 }
} catch (error) {
 console.error('Apollo agent mode failed, falling back to chat:', error);
}

// Fallbacks: search-grounded chat, then plain chat.
const attempts: Array<{ tools?: any[] }> = [
 { tools: [{ googleSearch: {} }] },
 {}
];

for (let i = 0; i < attempts.length; i++) {
 try {
 const responseStream = await client.models.generateContentStream({
 model: 'gemini-2.5-flash',
 contents,
 config: {
 systemInstruction,
 temperature: 0.7,
 ...attempts[i]
 }
 });
 }
}

```

```

 let emitted = false;
 for await (const chunk of responseStream) {
 if (chunk.text) {
 emitted = true;
 onChunk(chunk.text);
 }
 }
 if (emitted) return; // success
 } catch (error) {
 console.error(`Apollo Chat Error (attempt ${i + 1}):`, error);
 }
}

onChunk("Woof... I couldn't reach my brain just now. Give it a few seconds and ask again – if it keeps
happening, the daily free AI quota may be used up.");
};

export const fetchGeminiNews = async (city: string): Promise<NewsArticle[]> => {
 const client = getClient();
 if (!client) return [];

 try {
 const response = await client.models.generateContent({
 model: 'gemini-2.5-flash',
 contents: [{
 role: 'user',
 parts: [{
 text: `Perform a Google Search to find 3 distinct, high-quality, and recent travel ar
icles or guides for ${city}. Return RAW JSON array. Fields: title, source, publishedAt, url, imageUrl
(empty string).`
 }]
 }],
 config: { tools: [{ googleSearch: {} }] }
 });

 let text = response.text;
 if (!text) return [];
 text = text.replace(/```json/g, '').replace(/```/g, '').trim();
 const articles = JSON.parse(text);
 return articles.map((a: any, i: number) => ({
 id: `ai-news-${i}`,
 title: a.title,
 source: a.source || "City Guide",
 url: a.url || "#",
 publishedAt: a.publishedAt || "Recently",
 imageUrl: `https://source.unsplash.com/400x300/?${encodeURIComponent(city)},travel&sig=${i}`
 }));
 } catch (e) {
 console.error("Gemini News Error:", e);
 return [];
 }
};

```



```

export const getBudgetPlan = async (destination: string, days: number, travelers: number) => {
 const client = getClient();
 if (!client) return null;

 try {
 const response = await client.models.generateContent({
 model: 'gemini-2.5-flash',
 contents: [{
 role: 'user',
 parts: [{
 text: `Create a realistic travel budget for ${travelers} people visiting ${destination} for ${days} days. Return a RAW JSON object with estimated costs (numbers only) for these exact keys: "flight", "hotel", "food", "attraction", "other".`
 }]
 }],
 config: { responseType: "application/json" }
 });
 const text = response.text;
 if (!text) return null;
 return JSON.parse(text);
 } catch (e) {
 console.error("Budget Plan Error:", e);
 return null;
 }
};

```

```

export const generateAiNote = async (trip: any) => {
 const client = getClient();
 if (!client) return null;
 try {
 const flightsStr = trip.flights?.map((f: any) => `${f.arrival_airport} || 'TBD'`).join(', ') || 'Unknown destinations';
 const totalBudget = (trip.budget_categories || []).reduce((acc: number, curr: any) => acc + (curr.planned || 0), 0);
 const budgetDetails = (trip.budget_categories || []).map((b: any) => `${b.label}: $${b.planned}`)
 .join(', ');

 const response = await client.models.generateContent({
 model: 'gemini-2.5-flash',
 contents: [{
 role: 'user',
 parts: [{
 text: `Generate a helpful, creative, and cute travel insight for the trip "${trip.name}".

 Destinations involved: ${flightsStr}.
 Total Budget: $${totalBudget}.
 Budget Breakdown: ${budgetDetails || 'None yet'}.

 Return a JSON object with two fields: "title" (a catchy header) and "content" (a 3-4 sentence useful tip about places to save money, a cute insight about the destination, or budget optimization advice based on their spending).`
 }]
 }],
 config: { responseType: "application/json" }
 });
 }
};

```

```

 });
 const text = response.text;
 if (!text) return null;
 return JSON.parse(text);
 } catch (e) {
 console.error("AI Note Error", e);
 return null;
 }
}

export const generateSpeech = async (text: string): Promise<string | null> => {
 const client = getClient();
 if (!client) return null;
 try {
 const response = await client.models.generateContent({
 model: "gemini-2.5-flash-preview-tts",
 contents: [{ parts: [{ text }] }],
 config: {
 responseModalities: [Modality.AUDIO],
 speechConfig: { voiceConfig: { prebuiltVoiceConfig: { voiceName: "Puck" } } }
 }
 });
 const audioData = response.candidates?.[0]?.content?.parts?.[0]?.inlineData?.data;
 return audioData ? `data:audio/mp3;base64,${audioData}` : null;
 } catch (e) {
 console.error("TTS Error:", e);
 return null;
 }
};

export const generateTripStory = async (trip: any): Promise<string | null> => {
 const client = getClient();
 if (!client) return null;
 try {
 const flightsStr = trip.flights?.map((f: any) => `${f.airline} ${f.flight_number} on ${f.flight_date} from ${f.departure_airport} || 'TBD' to ${f.arrival_airport} || 'TBD'`).join(', ') || 'No flights booked yet.';

 const response = await client.models.generateContent({
 model: 'gemini-2.5-flash',
 contents: [{
 role: 'user',
 parts: [{
 text: `You are Apollo, a creative travel AI. Write a detailed, story-like summary for a trip named "${trip.name}".

Here are the trip details:
- Flights: ${flightsStr}
- Budget Items Count: ${trip.budget_categories?.length || 0}
- Notes Count: ${trip.notes?.length || 0}

Write a beautiful, exciting 3-4 paragraph narrative about this upcoming journey, formatted in Markdown. I include some tailored advice based on the destinations if known. Be very descriptive and paint a picture of the experience.`
]
 }]
 });
 }
};

```

```

 }}
 }}
 });
 return response.text || null;
} catch (e) {
 console.error("Trip Story Error", e);
 return null;
}
};

export const fetchFutureFlightFromGemini = async (flightNumber: string, date: string): Promise<Flight[]>
=> {
 const client = getClient();
 if (!client) return [];
 try {
 const response = await client.models.generateContent({
 model: 'gemini-2.5-flash',
 contents: [{
 role: 'user',
 parts: [{
 text: `Use Google Search to find the flight schedule for ${flightNumber} on or around
${date}.

 If it's too far in the future, predict it based on standard daily schedules.
 Return a single JSON object (NOT an array) representing the flight with these exact keys:

 "ident": "${flightNumber}",
 "airline": "Name of Airline",
 "departureAirport": "Origin Airport Code (e.g. ATL)",
 "arrivalAirport": "Destination Airport Code",
 "departureTime": "ISO 8601 string for departure on ${date}",
 "arrivalTime": "ISO 8601 string for arrival",
 "aircraft": "Aircraft type if known, or 'TBD'"
 Do not use markdown blocks, return RAW JSON.`
 }}
]},
 config: { responseType: "application/json" }
 });

 let text = response.text || "";
 if (!text) return [];
 text = text.replace(/```(?:json)?/gi, '').trim();
 const f = JSON.parse(text);

 let durationMinutes: number | null = null;
 if (f.departureTime && f.arrivalTime) {
 const dep = new Date(f.departureTime).getTime();
 const arr = new Date(f.arrivalTime).getTime();
 if (arr > dep) durationMinutes = Math.round((arr - dep) / 60000);
 }

 const flight: Flight = {
 id: `gemini-${Date.now()}`,
 ident: f.ident || flightNumber,
 flightNumber: f.ident || flightNumber,

```

```

 airline: f.airline || "Unknown",
 status: "Scheduled (Future)" as any,
 departureAirport: f.departureAirport || "TBD",
 arrivalAirport: f.arrivalAirport || "TBD",
 departureTime: f.departureTime || "",
 arrivalTime: f.arrivalTime || "",
 estimatedDepartureTime: "",
 actualDepartureTime: "",
 estimatedArrivalTime: "",
 actualArrivalTime: "",
 gate: "",
 terminal: "",
 gateDestination: "",
 terminalDestination: "",
 aircraft: f.aircraft || "TBD",
 progress: 0,
 delayMinutes: 0,
 durationMinutes
 };
 return [flight];
} catch (e) {
 console.error("Future Flight Gemini Error:", e);
 return [];
}
};

export const generateBudgetInsight = async (trip: any): Promise<string> => {
 const client = getClient();
 if (!client) return "Apollo couldn't connect. API Key missing.";

 const destination = trip.destination || "Unknown Destination";
 const budgetLimit = trip.budget_limit || 0;
 const durationDays = trip.duration_days || 1;
 const travelers = trip.travelers_count || 1;

 const items = trip.budget_categories || [];
 const totalSpent = items.reduce((sum: number, item: any) => sum + (item.planned || 0), 0);
 const remaining = budgetLimit - totalSpent;

 const itemsStr = items.map((i: any) => `-${i.label}: ${i.planned}`).join("\n");

 const prompt = `You are Apollo AI, a highly intelligent and friendly travel companion.
The user is planning a trip to ${destination} for ${durationDays} days with ${travelers} traveler(s).
Their total budget is ${budgetLimit}. They have spent ${totalSpent} so far, leaving ${remaining}.

Here are their specific expenses:
${itemsStr || "No expenses added yet."}

Analyze this budget considering the typical costs in ${destination}.
- Are they over or under budget?
- Will ${remaining} be enough for the remaining days for ${travelers} people?
- What typical costs in ${destination} (like food or transport) should they watch out for?

Provide a concise, helpful, and slightly playful insight (1-2 paragraphs max).`;

```

```
 try {
 const model = client.models;
 const response = await model.generateContent({
 model: 'gemini-2.5-flash',
 contents: prompt
 });
 return response.text || "Apollo couldn't generate an insight right now.";
 } catch (error) {
 console.error("Budget insight error:", error);
 return "Apollo encountered an error generating your budget insight.";
 }
 }
};
```

```
=====
FILE: services/mockService.ts
=====
```

```
import { Flight, FlightStatus, Airport, Place, Weather, DelayInsight, FlightSchedule, GateInfo, Australia
 nAirspace } from '../types';
```

```
export const AIRLINE_CODES = [
 { code: "DL", icao: "DAL", name: "Delta Air Lines" },
 { code: "AA", icao: "AAL", name: "American Airlines" },
 { code: "UA", icao: "UAL", name: "United Airlines" },
 { code: "WN", icao: "SWA", name: "Southwest Airlines" },
 { code: "B6", icao: "JBU", name: "JetBlue Airways" },
 { code: "AS", icao: "ASA", name: "Alaska Airlines" },
 { code: "F9", icao: "FFT", name: "Frontier Airlines" },
 { code: "HA", icao: "HAL", name: "Hawaiian Airlines" },
 { code: "AC", icao: "ACA", name: "Air Canada" },
 { code: "WS", icao: "WJA", name: "WestJet" },
 { code: "BA", icao: "BAW", name: "British Airways" },
 { code: "LH", icao: "DLH", name: "Lufthansa" },
 { code: "AF", icao: "AFR", name: "Air France" },
 { code: "EK", icao: "UAE", name: "Emirates" },
 { code: "QF", icao: "QFA", name: "Qantas" }
];
```

```
import { EXTENDED_AIRPORTS } from './airportsData';
```

```
const BASE_AIRPORTS = [
 { code: "ATL", city: "Atlanta", name: "Hartsfield-Jackson", lat: 33.64, lng: -84.42 },
 { code: "LAX", city: "Los Angeles", name: "Los Angeles Intl", lat: 33.94, lng: -118.40 },
 { code: "JFK", city: "New York", name: "John F. Kennedy", lat: 40.64, lng: -73.77 },
 { code: "ORD", city: "Chicago", name: "O'Hare", lat: 41.97, lng: -87.90 },
 { code: "LHR", city: "London", name: "Heathrow", lat: 51.47, lng: -0.46 },
 { code: "SFO", city: "San Francisco", name: "San Francisco Intl", lat: 37.62, lng: -122.37 },
 { code: "MIA", city: "Miami", name: "Miami Intl", lat: 25.79, lng: -80.28 },
 { code: "SEA", city: "Seattle", name: "Tacoma Intl", lat: 47.45, lng: -122.30 },
 { code: "DXB", city: "Dubai", name: "Dubai Intl", lat: 25.25, lng: 55.36 },
 { code: "HND", city: "Tokyo", name: "Haneda", lat: 35.54, lng: 139.77 },
 { code: "CDG", city: "Paris", name: "Charles de Gaulle", lat: 49.00, lng: 2.55 },
 { code: "AMS", city: "Amsterdam", name: "Schiphol", lat: 52.31, lng: 4.76 },
 { code: "SIN", city: "Singapore", name: "Changi", lat: 1.36, lng: 103.99 },
 { code: "SYD", city: "Sydney", name: "Kingsford Smith", lat: -33.93, lng: 151.17 },
```

```
{ code: "DFW", city: "Dallas", name: "Dallas/Fort Worth", lat: 32.89, lng: -97.04 },
{ code: "DEN", city: "Denver", name: "Denver Intl", lat: 39.85, lng: -104.67 },
{ code: "LAS", city: "Las Vegas", name: "Harry Reid", lat: 36.08, lng: -115.15 },
{ code: "CLT", city: "Charlotte", name: "Charlotte Douglas", lat: 35.21, lng: -80.94 },
{ code: "MCO", city: "Orlando", name: "Orlando Intl", lat: 28.43, lng: -81.30 },
{ code: "EWR", city: "Newark", name: "Newark Liberty", lat: 40.68, lng: -74.17 },
{ code: "PHX", city: "Phoenix", name: "Sky Harbor", lat: 33.43, lng: -112.00 },
{ code: "IAH", city: "Houston", name: "George Bush", lat: 29.99, lng: -95.33 },
{ code: "BOS", city: "Boston", name: "Logan Intl", lat: 42.36, lng: -71.00 }
];

const AIRPORTS = [...BASE_AIRPORTS, ...EXTENDED_AIRPORTS];

const DELAY_REASONS: DelayInsight[] = [
 { reason: "Late Incoming Aircraft", confidence: 85, description: "Your plane is arriving late from its previous trip." },
 { reason: "Weather", confidence: 90, description: "Thunderstorms in the flight path." },
 { reason: "ATC Volume", confidence: 60, description: "Air traffic control has paused departures due to volume." },
 { reason: "Crew Availability", confidence: 95, description: "Awaiting flight crew member." }
];

export const generateMockFlights = (count: number): Flight[] => {
 const airlines = ["Delta", "American", "United", "JetBlue", "Southwest", "Emirates", "British Airways", "Lufthansa", "Air France", "Qantas"];
 const statuses = Object.values(FlightStatus);

 return Array.from({ length: count }).map((_, i) => {
 const dep = AIRPORTS[Math.floor(Math.random() * AIRPORTS.length)];
 let arr = AIRPORTS[Math.floor(Math.random() * AIRPORTS.length)];
 while (arr.code === dep.code) {
 arr = AIRPORTS[Math.floor(Math.random() * AIRPORTS.length)];
 }

 let status = statuses[Math.floor(Math.random() * statuses.length)];
 if (Math.random() > 0.7) status = FlightStatus.OnTime;
 const isDelayed = status === FlightStatus.Delayed;
 const progress = status === FlightStatus.EnRoute ? Math.floor(Math.random() * 80) + 10 : (status === FlightStatus.Landed ? 100 : 0);
 const airline = airlines[Math.floor(Math.random() * airlines.length)];
 const flightNum = Math.floor(Math.random() * 9000) + 1000;

 return {
 id: `flight-${i}`,
 ident: `FL${flightNum}`,
 flightNumber: `${airline.substring(0, 2).toUpperCase()}${flightNum}`,
 airline: airline,
 status,
 departureAirport: dep.code,
 arrivalAirport: arr.code,
 departureTime: new Date(Date.now() + Math.random() * 86400000).toISOString(),
 arrivalTime: new Date(Date.now() + Math.random() * 172800000).toISOString(),
 gate: `${['A', 'B', 'C', 'T'][Math.floor(Math.random() * 4)]}${Math.floor(Math.random() * 30) + 1}`,
 terminal: `${Math.floor(Math.random() * 5) + 1}`,
 };
 });
}
```

```
 aircraft: ["Boeing 737-MAX", "Airbus A321neo", "Boeing 787-9", "Airbus A350-1000", "Boeing 777-300ER",
][Math.floor(Math.random() * 5)],
 tailNumber: `N${Math.floor(Math.random() * 900) + 100}XY`,
 progress,
 delayMinutes: isDelayed ? Math.floor(Math.random() * 120) + 15 : 0,
 price: Math.floor(Math.random() * 600) + 150,
 insight: isDelayed ? DELAY_REASONS[Math.floor(Math.random() * DELAY_REASONS.length)] : undefined,
 websiteUrl: "https://www.google.com/travel/flights",
 };
});
};

export const getMockFlightById = (ident: string): Flight | null => {
 const mockList = generateMockFlights(1);
 const flight = mockList[0];
 return {
 ...flight,
 id: ident,
 flightNumber: ident,
 airline: "Simulated Airline",
 status: FlightStatus.EnRoute,
 progress: 45
 };
};

export const getAirportSuggestions = (query: string): string[] => {
 if (!query) return [];
 const q = query.toLowerCase();
 return AIRPORTS
 .filter(a => a.code.toLowerCase().includes(q) || a.city.toLowerCase().includes(q) || a.name.toLowerCase().includes(q))
 .slice(0, 5)
 .map(a => `${a.code} - ${a.city}`);
};

export const getAirlineSuggestions = (query: string): string[] => {
 if (!query || query.length > 3) return []; // only suggest if they are typing the prefix
 const q = query.toLowerCase();
 return AIRLINE_CODES
 .filter(a => a.code.toLowerCase().startsWith(q) || a.icao.toLowerCase().startsWith(q))
 .slice(0, 5)
 .map(a => `${a.code} - ${a.name}`);
};

export const resolveAirportCode = (query: string): string | null => {
 if (!query) return null;
 const q = query.toLowerCase().trim();
 if (q.length === 3 || q.length === 4) return q.toUpperCase();

 const match = AIRPORTS.find(a =>
 a.city.toLowerCase() === q ||
 a.name.toLowerCase() === q ||
 a.city.toLowerCase().includes(q)
);
};
```

```
 return match ? match.code : null;
 };

export const getAirportCoords = (code: string) => {
 const c = (code || '').toUpperCase();
 let airport = AIRPORTS.find(a => a.code === c);
 // ICAO fallback: US airports are the IATA code with a K prefix (KATL -> ATL)
 if (!airport && c.length === 4 && c.startsWith('K')) {
 airport = AIRPORTS.find(a => a.code === c.slice(1));
 }
 return airport ? { lat: airport.lat, lng: airport.lng } : { lat: 0, lng: 0 };
};

export const getMockAirports = (): Airport[] => {
 return AIRPORTS.map(a => ({
 code: a.code,
 name: a.name,
 city: a.city,
 state: "INTL",
 tsawaitMinutes: Math.floor(Math.random() * 45) + 2,
 isPreCheckOpen: Math.random() > 0.1
 }));
};

export const getMockWeather = (city: string): Weather => ({
 city,
 temp: Math.floor(Math.random() * 40) + 50,
 condition: ["Sunny", "Partly Cloudy", "Rainy", "Clear"][Math.floor(Math.random() * 4)],
 humidity: Math.floor(Math.random() * 50) + 30,
 wind: Math.floor(Math.random() * 15) + 2
});

export const getMockPlaces = (): Place[] => [
 {
 id: '1', name: 'Terminal Bistro', category: 'Food', rating: 4.5, priceLevel: 2,
 priceEstimate: 45, priceDisplay: "$20-45", image: 'https://picsum.photos/200/200?random=1',
 coordinates: { x: 30, y: 40 }, description: "Upscale dining with a view of the runway.", websiteUrl: ""
 },
 {
 id: '2', name: 'Skyline Hotel', category: 'Hotel', rating: 4.2, priceLevel: 3,
 priceEstimate: 210, priceDisplay: "$50-90", image: 'https://picsum.photos/200/200?random=2',
 coordinates: { x: 70, y: 20 }, description: "Luxury stay for the weary traveler.", websiteUrl: ""
 },
 {
 id: '3', name: 'Aviation Museum', category: 'Attraction', rating: 4.8, priceLevel: 1,
 priceEstimate: 25, priceDisplay: "$10-20", image: 'https://picsum.photos/200/200?random=3',
 coordinates: { x: 50, y: 80 }, description: "Historical aircrafts and flight simulators.", websiteUrl: ""
 },
];

export const getTransitRates = (city: string) => {
```



```
 const baseRate = city.length * 2 + 10;
 return {
 uber: { low: baseRate, high: baseRate + 15 },
 lyft: { low: baseRate - 2, high: baseRate + 12 },
 public: 2.50
 };
 };

export const getMockSchedules = (): FlightSchedule[] => [
 { ident: "DL123", departure: new Date().toISOString(), arrival: new Date().toISOString(), origin: "ATL", destination: "LHR" }
];

export const getMockGates = (airportCode: string): GateInfo[] => [
 { gate: "A1", status: "Open", flight: "DL123" },
 { gate: "B4", status: "Closed", flight: null }
];

export const getMockAirspace = (): AustralianAirspace => ({
 activeFlights: 120,
 restrictions: ["R345 Active"],
 weatherAlerts: ["Turbulence over SYD"]
});

// US Cities/States for budget & note autocomplete
const US_LOCATIONS = [
 { city: 'New York', state: 'NY', zip: '10001' },
 { city: 'Los Angeles', state: 'CA', zip: '90001' },
 { city: 'Chicago', state: 'IL', zip: '60601' },
 { city: 'Houston', state: 'TX', zip: '77001' },
 { city: 'Phoenix', state: 'AZ', zip: '85001' },
 { city: 'Philadelphia', state: 'PA', zip: '19101' },
 { city: 'San Antonio', state: 'TX', zip: '78201' },
 { city: 'San Diego', state: 'CA', zip: '92101' },
 { city: 'Dallas', state: 'TX', zip: '75201' },
 { city: 'San Jose', state: 'CA', zip: '95101' },
 { city: 'Austin', state: 'TX', zip: '73301' },
 { city: 'Jacksonville', state: 'FL', zip: '32099' },
 { city: 'Fort Worth', state: 'TX', zip: '76101' },
 { city: 'Columbus', state: 'OH', zip: '43085' },
 { city: 'Charlotte', state: 'NC', zip: '28201' },
 { city: 'Indianapolis', state: 'IN', zip: '46201' },
 { city: 'San Francisco', state: 'CA', zip: '94101' },
 { city: 'Seattle', state: 'WA', zip: '98101' },
 { city: 'Denver', state: 'CO', zip: '80201' },
 { city: 'Nashville', state: 'TN', zip: '37201' },
 { city: 'Oklahoma City', state: 'OK', zip: '73101' },
 { city: 'Las Vegas', state: 'NV', zip: '89101' },
 { city: 'Portland', state: 'OR', zip: '97201' },
 { city: 'Miami', state: 'FL', zip: '33101' },
 { city: 'Atlanta', state: 'GA', zip: '30301' },
 { city: 'Tampa', state: 'FL', zip: '33601' },
 { city: 'Orlando', state: 'FL', zip: '32801' },
 { city: 'Minneapolis', state: 'MN', zip: '55401' },
];
```

```

 { city: 'Honolulu', state: 'HI', zip: '96801' },
 { city: 'Paris', state: 'France', zip: '75000' },
 { city: 'London', state: 'UK', zip: 'EC1A' },
 { city: 'Tokyo', state: 'Japan', zip: '100-0001' },
 { city: 'Dubai', state: 'UAE', zip: '00000' },
 { city: 'Rome', state: 'Italy', zip: '00100' },
 { city: 'Barcelona', state: 'Spain', zip: '08001' },
 { city: 'Amsterdam', state: 'Netherlands', zip: '1011' },
 { city: 'Sydney', state: 'Australia', zip: '2000' },
 { city: 'Singapore', state: 'Singapore', zip: '018956' },
 { city: 'Cancun', state: 'Mexico', zip: '77500' },
 { city: 'Toronto', state: 'Canada', zip: 'M5V' },
];

export const getLocationSuggestions = (query: string): { city: string; state: string; zip: string }[] =>
{
 if (!query || query.length < 2) return [];
 const q = query.toLowerCase();
 return US_LOCATIONS.filter(loc =>
 loc.city.toLowerCase().includes(q) ||
 loc.state.toLowerCase().includes(q) ||
 loc.zip.includes(q)
).slice(0, 6);
};

```

```

=====
FILE: services/tripService.ts
=====

```

```

import { db } from '../firebaseClient';
import {
 collection,
 doc,
 getDoc,
 getDocs,
 setDoc,
 addDoc,
 deleteDoc,
 updateDoc,
 query,
 where,
 orderBy
} from 'firebase/firestore';
import { Trip, TripFlight, Note, BudgetCategory, Pass } from '../types';

const MOCK_STORAGE_KEY = 'urtc_mock_trips';

// Helper to get local mock trips
const getMockTrips = (): Trip[] => {
 const data = localStorage.getItem(MOCK_STORAGE_KEY);
 return data ? JSON.parse(data) : [];
};

// Helper to save local mock trips
const saveMockTrips = (trips: Trip[]) => {

```

```
 localStorage.setItem(MOCK_STORAGE_KEY, JSON.stringify(trips));
 };

export const fetchTrips = async (userId: string): Promise<Trip[]> => {
 if (userId === 'guest' || userId.startsWith('code-')) {
 return getMockTrips().filter(t => t.user_id === userId);
 }

 try {
 const tripsRef = collection(db, 'trips');
 const q = query(tripsRef, where('user_id', '=', userId));
 const snapshot = await getDocs(q);

 const trips: Trip[] = [];
 snapshot.forEach(doc => {
 trips.push({ id: doc.id, ...doc.data() } as Trip);
 });

 // Sort by created_at descending (approximate if we don't have indexes yet)
 return trips.sort((a, b) => {
 const dateA = a.created_at ? new Date(a.created_at).getTime() : 0;
 const dateB = b.created_at ? new Date(b.created_at).getTime() : 0;
 return dateB - dateA;
 });
 } catch (error) {
 console.error('Error fetching trips:', error);
 return [];
 }
};

export const createTrip = async (userId: string, name: string): Promise<Trip | null> => {
 const newTrip: any = {
 user_id: userId,
 name,
 notes: [],
 budget_categories: [],
 passes: [],
 flights: [],
 created_at: new Date().toISOString()
 };

 if (userId === 'guest' || userId.startsWith('code-')) {
 const trip = { ...newTrip, id: `mock-trip-${Date.now()}` };
 const trips = getMockTrips();
 saveMockTrips([trip, ...trips]);
 return trip;
 }

 try {
 const docRef = await addDoc(collection(db, 'trips'), newTrip);
 return { id: docRef.id, ...newTrip };
 } catch (error: any) {
 console.error('Error creating trip:', error);
 alert(`Error: ${error.message}`);
 }
};
```

```
 return null;
 }
};

export const deleteTrip = async (tripId: string): Promise<boolean> => {
 if (tripId.startsWith('mock-trip-')) {
 const trips = getMockTrips().filter(t => t.id !== tripId);
 saveMockTrips(trips);
 return true;
 }

 try {
 await deleteDoc(doc(db, 'trips', tripId));
 return true;
 } catch (error) {
 console.error('Error deleting trip:', error);
 return false;
 }
};

export const updateTrip = async (tripId: string, updates: Partial<Trip>): Promise<boolean> => {
 if (tripId.startsWith('mock-trip-')) {
 const trips = getMockTrips();
 const index = trips.findIndex(t => t.id === tripId);
 if (index > -1) {
 trips[index] = { ...trips[index], ...updates };
 saveMockTrips(trips);
 return true;
 }
 return false;
 }

 try {
 await updateDoc(doc(db, 'trips', tripId), updates);
 return true;
 } catch (error) {
 console.error('Error updating trip:', error);
 return false;
 }
};

export const addFlightToTrip = async (userId: string, tripId: string, flightNumber: string, date: string,
 airline?: string, departure?: string, arrival?: string): Promise<boolean> => {
 const newFlight: any = {
 id: `flight-${Date.now()}`,
 trip_id: tripId,
 user_id: userId,
 flight_number: flightNumber,
 airline,
 flight_date: date,
 departure_airport: departure,
 arrival_airport: arrival,
 status: 'Scheduled (Awaiting Live Telemetry)'
 };
};
```

```
if (tripId.startsWith('mock-trip-')) {
 const trips = getMockTrips();
 const trip = trips.find(t => t.id === tripId);
 if (trip) {
 if (!trip.flights) trip.flights = [];
 trip.flights.push(newFlight);
 saveMockTrips(trips);
 return true;
 }
 return false;
}

try {
 // Fetch current trip to append flight
 const tripRef = doc(db, 'trips', tripId);
 const tripSnap = await getDoc(tripRef);
 if (tripSnap.exists()) {
 const data = tripSnap.data();
 const flights = data.flights || [];
 flights.push(newFlight);
 await updateDoc(tripRef, { flights });
 return true;
 }
 return false;
} catch (error) {
 console.error('Error adding flight to trip:', error);
 return false;
}
};

export const deleteFlightFromTrip = async (flightId: string): Promise<boolean> => {
 if (flightId.startsWith('flight-')) {
 // Mock trips might also have 'flight-' or 'mock-flight-' depending on generation
 const trips = getMockTrips();
 let changed = false;
 for (const trip of trips) {
 if (trip.flights) {
 const initLen = trip.flights.length;
 trip.flights = trip.flights.filter(f => f.id !== flightId);
 if (trip.flights.length !== initLen) changed = true;
 }
 }
 if (changed) saveMockTrips(trips);
 // We'll also try Firestore just in case
 }

 try {
 // For Firestore, we have to find the trip that contains this flight.
 // This is inefficient but works for now since we denormalized flights into the trip document.
 const snapshot = await getDocs(collection(db, 'trips'));
 let found = false;
 for (const tripDoc of snapshot.docs) {
 const data = tripDoc.data();
 }
 }
}
```

```

 const flights = data.flights || [];
 const newFlights = flights.filter((f: any) => f.id !== flightId);
 if (flights.length !== newFlights.length) {
 await updateDoc(doc(db, 'trips', tripDoc.id), { flights: newFlights });
 found = true;
 break;
 }
 }
 return found;
} catch (error) {
 console.error('Error deleting flight:', error);
 return false;
}
};

```

```

=====
FILE: i18n/context.tsx
=====

```

```

import React, { createContext, useContext, useState, useEffect, ReactNode } from 'react';
import { translations, Language } from '../translations';

```

```

interface LanguageContextType {
 language: Language;
 setLanguage: (lang: Language) => void;
 t: (key: string) => string;
}

```

```

const LanguageContext = createContext<LanguageContextType>({
 language: 'en',
 setLanguage: () => {},
 t: (key: string) => key,
});

```

```

export const LanguageProvider = ({ children }: { children: ReactNode }) => {
 const [language, setLanguageState] = useState<Language>('en');

```

```

 useEffect(() => {
 const saved = localStorage.getItem('urtc_language') as Language;
 if (saved && translations[saved]) {
 setLanguageState(saved);
 } else {
 // Auto-detect browser language if possible
 const browserLang = navigator.language.split('-')[0] as Language;
 if (translations[browserLang]) {
 setLanguageState(browserLang);
 }
 }
 }, []);

```

```

const setLanguage = (lang: Language) => {
 setLanguageState(lang);
 localStorage.setItem('urtc_language', lang);
};

```

```
const t = (path: string): string => {
 const keys = path.split('.');
 let current: any = translations[language];

 for (const key of keys) {
 if (current[key] === undefined) {
 // Fallback to English if translation is missing
 let fallback: any = translations['en'];
 for (const fKey of keys) {
 if (fallback[fKey] === undefined) return path;
 fallback = fallback[fKey];
 }
 return fallback;
 }
 current = current[key];
 }
 return current;
};

return (
 <LanguageContext.Provider value={{ language, setLanguage, t }}>
 {children}
 </LanguageContext.Provider>
);
};
```

```
export const useLanguage = () => useContext(LanguageContext);
```

```
=====
FILE: i18n/translations.ts
=====
```

```
export const translations = {
 en: {
 tabs: {
 home: "Home",
 explore: "Explore",
 flights: "Flights",
 wander: "Wander",
 apollo: "Apollo AI",
 plans: "Plans",
 about: "About"
 },
 common: {
 search: "Search",
 save: "Save",
 cancel: "Cancel",
 loading: "Loading..."
 }
 },
 es: {
 tabs: {
 home: "Inicio",
 explore: "Explorar",
 flights: "Vuelos",
```

```
wander: "Wander",
apollo: "Apollo AI",
plans: "Planes",
about: "Acerca de"
},
common: {
 search: "Buscar",
 save: "Guardar",
 cancel: "Cancelar",
 loading: "Cargando..."
}
},
fr: {
 tabs: {
 home: "Accueil",
 explore: "Explorer",
 flights: "Vols",
 wander: "Wander",
 apollo: "Apollo AI",
 plans: "Projets",
 about: "À propos"
 },
 common: {
 search: "Rechercher",
 save: "Enregistrer",
 cancel: "Annuler",
 loading: "Chargement..."
 }
},
de: {
 tabs: {
 home: "Start",
 explore: "Entdecken",
 flights: "Flüge",
 wander: "Wander",
 apollo: "Apollo AI",
 plans: "Pläne",
 about: "Über uns"
 },
 common: {
 search: "Suchen",
 save: "Speichern",
 cancel: "Abbrechen",
 loading: "Wird geladen..."
 }
},
zh: {
 tabs: {
 home: "■■■",
 explore: "■■■",
 flights: "■■■",
 wander: "Wander",
 apollo: "Apollo AI",
 plans: "■■■",
```



```

 about: "■■■"
 },
 common: {
 search: "■■■",
 save: "■■■",
 cancel: "■■■",
 loading: "■■■■..."
 }
}
};

```

```
export type Language = keyof typeof translations;
```

```
=====
FILE: api/webhook.ts
=====
// Moved to functions/index.js (Firebase Cloud Function "stripewebhook").
// This Vercel-era file is retired and intentionally left empty.
export {};
```

```
=====
FILE: functions/index.js
=====
// urtc functions v1.1.4 - aeroapi secret binding
const { onRequest } = require("firebase-functions/v2/https");
```

```
const AERO_ORIGIN = "https://aeroapi.flightaware.com";

// Proxies /aeroapi/** requests from Firebase Hosting to FlightAware AeroAPI.
// The API key is taken from the AEROAPI_KEY env var if set (recommended),
// otherwise the client-supplied x-apikey header is forwarded.
// invoker: "public" – hosting rewrites arrive unauthenticated; without this
// Cloud Run returns 403 before the function even runs.
```

```
// Shared AeroAPI cache
// Every user request passes through here; identical requests within the
// TTL window are served from cache and never billed by FlightAware.
const adminForCache = require("firebase-admin");
if (!adminForCache.apps.length) adminForCache.initializeApp();
const crypto = require("crypto");
```

```
const memCache = new Map(); // key -> { exp, body, contentType }
const MEM_CACHE_MAX = 500;
```

```
// How long each kind of data stays fresh (ms)
const ttlFor = (url) => {
 if (url.includes("/alerts")) return 0; // never cache account alert config
 if (url.includes("/account/")) return 0;
 if (url.includes("/history/")) return 6 * 3600e3; // history never changes
 if (url.includes("/schedules/")) return 12 * 3600e3; // published schedules
 if (url.match(/\airports\[^\]+\/routes\/\//)) return 12 * 3600e3;
 if (url.match(/\airports\[^\]+\/weather/)) return 15 * 60e3;
 if (url.match(/\flights\[^\]+\/(position|track)/)) return 20e3; // live positions stay hot
 if (url.includes("/flights/search")) return 30e3;
}
```

```

 if (url.match(/\airports\[^\]+\flights/)) return 60e3; // shared airport boards
 if (url.includes("/airports/delays")) return 120e3;
 if (url.match(/\airports\[^\]+\delays/)) return 120e3;
 if (url.includes("/operators/")) return 120e3;
 if (url.match(/\aircraft\//)) return 24 * 3600e3; // owners/types are static
 if (url.match(/\airports\[^\]+$/)) return 24 * 3600e3; // airport info is static
 return 60e3; // default: flight lookups etc.
 };

const cacheGet = async (key) => {
 const hit = memCache.get(key);
 if (hit && hit.exp > Date.now()) return hit;
 if (hit) memCache.delete(key);
 try {
 const doc = await adminForCache.firestore().collection("aeroapi_cache").doc(key).get();
 if (doc.exists) {
 const d = doc.data();
 if (d.exp > Date.now()) {
 memCache.set(key, d);
 return d;
 }
 }
 } catch (e) { /* cache read failure is never fatal */ }
 return null;
};

const cacheSet = (key, body, contentType, ttl) => {
 const entry = { exp: Date.now() + ttl, body, contentType };
 if (memCache.size >= MEM_CACHE_MAX) memCache.delete(memCache.keys().next().value);
 memCache.set(key, entry);
 // Firestore docs cap at ~1MB — persist only what fits
 if (body.length < 900000) {
 adminForCache.firestore().collection("aeroapi_cache").doc(key).set(entry).catch(() => {});
 }
};

exports.aeroapi = onRequest({ region: "us-central1", cors: true, invoker: "public", secrets: ["AEROAPI_KEY"] }, async (req, res) => {
 // Diagnostic peephole: shows whether the env key is bound (never reveals the key)
 if (req.originalUrl.includes("__diag")) {
 const k = process.env.AEROAPI_KEY || "";
 res.json({ hasEnvKey: k.length > 0, keyLength: k.length, keyPrefix: k.slice(0, 3), keySuffix: k.slice(-3), trimmedSame: k === k.trim() });
 return;
 }
 try {
 const url = AERO_ORIGIN + req.originalUrl;
 const ttl = ttlFor(req.originalUrl);
 const cacheKey = crypto.createHash("sha1").update(req.originalUrl).digest("hex");

 // Serve from the shared cache when possible (GET only)
 if (req.method === "GET" && ttl > 0) {
 const hit = await cacheGet(cacheKey);
 if (hit) {

```

[illegible]

```

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY || "");
let event;
try {
 event = stripe.webhooks.constructEvent(
 req.rawBody, // Firebase Functions provides the raw body for signature verification
 req.headers["stripe-signature"],
 process.env.STRIPE_WEBHOOK_SECRET || ""
);
} catch (err) {
 console.error("Webhook signature verification failed:", err.message);
 res.status(400).send(`Webhook Error: ${err.message}`);
 return;
}

if (event.type === "checkout.session.completed") {
 const session = event.data.object;
 const userId = session.client_reference_id;
 // Tier from metadata if set on the Payment Link; default to Diamond.
 // Legacy "Pro" values map to Diamond.
 let tier = session.metadata && session.metadata.tier;
 if (!tier || tier === "Pro") tier = "Diamond";
 if (!VALID_TIERS.includes(tier)) tier = "Diamond";

 if (userId) {
 try {
 await admin.firestore().collection("users").doc(userId).set(
 {
 tier,
 stripe_customer_id: session.customer || null,
 upgraded_at: admin.firestore.FieldValue.serverTimestamp(),
 },
 { merge: true }
);
 console.log(`Upgraded user ${userId} to ${tier}`);
 } catch (error) {
 console.error("Error updating user tier:", error);
 res.status(500).json({ error: "Database update failed" });
 return;
 }
 } else {
 console.warn("checkout.session.completed without client_reference_id – cannot upgrade anyone.");
 }
}

res.status(200).json({ received: true });
};

=====
FILE: public/privacy.html
=====
<!DOCTYPE html>
<html lang="en">
<head>

```

```
<meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Privacy Policy – ÜrTC by Cave Core Dynamics</title>
<style>body{font-family:-apple-system,BlinkMacSystemFont,'Segoe UI',Roboto,sans-serif;background:#0B0D10;
 color:#d1d5db;max-width:760px;margin:0 auto;padding:48px 24px;line-height:1.7}h1{color:#fff}h2{color:#F
 F5C1A;margin-top:32px;font-size:18px}a{color:#3AB0FF}.meta{color:#6b7280;font-size:13px}</style>
</head>
<body>
<h1>ÜrTC Privacy Policy</h1>
<p class="meta">Cave Core Dynamics™ · Effective July 2026 · Contact: admin@cavecoredynamics.org</p>

<h2>What we collect</h2>
<p>Account information (email, username) when you create an account; location data (with your permission)
 to show nearby places, weather, and air quality; trip data you create (flights, places, notes, budgets
); and payment status via Stripe (we never see or store card numbers).</p>

<h2>How we use it</h2>
<p>To provide the app's features: live flight tracking, nearby recommendations, trip planning, AI assista
 nce (Apollo), and subscription management. AI conversations are processed by Google Gemini to generate
 responses. Flight data is provided by third-party aviation data services; weather by OpenWeather; maps
 and places by Google Maps Platform.</p>

<h2>What we don't do</h2>
<p>We do not sell your personal data. We do not share your location with advertisers. Apollo conversatio
 ns are not used to train AI models by Cave Core Dynamics.</p>

<h2>Payments</h2>
<p>Subscriptions are processed by Stripe. Payment details are handled entirely by Stripe under their priv
 acy policy. We receive only your subscription status and a customer reference.</p>

<h2>Data storage & deletion</h2>
<p>Account and trip data are stored in Google Firebase (Firestore) in the United States. To delete your a
 ccount and all associated data, email admin@cavecoredynamic
 s.org from your account email and we will complete deletion within 30 days.</p>

<h2>Advertising</h2>
<p>Free-tier users may see advertisements. Diamond subscribers see no ads.</p>

<h2>Children</h2>
<p>ÜrTC is not directed at children under 13, and we do not knowingly collect data from them.</p>

<h2>Changes</h2>
<p>We'll update this page when our practices change. Material changes will be announced in the app.</p>
</body>
</html>
```